

CanvasApp — Demo Guide

1. **Code evidence** — file + dòng cần mở sẵn trong Visual Studio 2022
2. **Demo runtime** — thao tác chạy thật
3. **Bằng chứng quan sát** — log console / record DB / packet Wireshark giảng viên có thể thấy

Mục lục

- [A. Chuẩn bị \(làm 1 lần trước buổi demo\)](#)
- [B. Khởi động hệ thống](#)
- [C. Demo từng mục rubric \(10đ\)](#)
- [D. Phụ lục: Wireshark / AES / netstat / Troubleshooting / Checklist](#)

A. Chuẩn bị

A.1. Bộ công cụ cần mở sẵn

Công cụ	Mục đích	Mở bằng
Visual Studio 2022	Mở solution, build, debug, show code	Mở CanvasApp.sln
XAMPP Control Panel	Bật MySQL + (tùy chọn) Apache cho phpMyAdmin	Start menu → XAMPP
phpMyAdmin hoặc MySQL Workbench	Show database table + record	http://localhost/phpmyadmin
PowerShell / cmd	Chạy ipconfig , netstat , khởi .exe thủ công	Win + R → powershell
Wireshark (tùy chọn — extra credit I/O Network)	Bắt gói TCP để chứng minh truyền JSON qua mạng	wireshark.org (cài kèm Npcap , tick " WinPcap API-compatible Mode ")

A.2. Database

1. Mở **XAMPP** → Start **MySQL** (port 3306).
2. Vào phpMyAdmin → nếu chưa có DB [canvasapp](#):
 - Import [Database/schema.sql](#)
 - Import [Database/seed_data.sql](#) (tùy chọn — tạo 3 user demo + 2 room mẫu)
3. Kiểm tra có **7 bảng**: [users](#), [rooms](#), [room_members](#), [draw_actions](#), [canvas_snapshots](#), [chat_messages](#), [email_otp_codes](#).

Cần reset dữ liệu giữa buổi demo? Import [Database/reset.sql](#).

A.3. Build solution

- Trong Visual Studio: **Ctrl + Shift + B**. Output phải hiện **Build succeeded. 0 Error(s)**.
- Build xong các **.exe** nằm trong **bin/Debug/** của từng project — sẽ dùng ở phần B.2.

A.4. Cấu hình "Multiple Startup Projects"

Mục tiêu: 1 cú **F5** mở luôn 3 console (AuthServer + CanvasServer + LoadBalancer).

1. **Solution Explorer** → chuột phải **Solution 'CanvasApp'** → **Configure Startup Projects...**
2. Tick **Multiple startup projects** và set Action:

Project	Action
CanvasApp.AuthServer	Start
CanvasApp.Server	Start
CanvasApp.LoadBalancer	Start
CanvasApp.Client	None (mở thủ công sau)

3. Dùng mũi tên ▲/▼ để **đẩy LoadBalancer xuống cuối** (start sau cùng → khi LB lên thì 2 backend đã listening).
4. Trỏ Client về Load Balancer: mở [CanvasApp.Client/Network/Session.cs](#):

```
public const int AUTH_PORT    = 9000;    // đi qua LB
public const int CANVAS_PORT = 9000;    // đi qua LB
```

A.5. Cấu hình network cho demo qua LAN/VPN (chỉ khi demo cross-machine)

A.5.1. Cùng mạng LAN/Wi-Fi

- Trên **máy Server**: **ipconfig** → ghi nhớ **IPv4 Address** (ví dụ **192.168.1.42**).
- Đảm bảo các listener bind **IPAddress.Any** (đã code sẵn, không cần sửa) — xem [Server/Program.cs](#), [AuthServer/AuthServer.cs](#), [LoadBalancer/LoadBalancer.cs](#).
- Trên **máy Client**: sửa **Session.cs** → **LB_HOST = "192.168.1.42"** → rebuild.

A.5.2. Khác mạng — qua VPN ảo

- Cài **Radmin VPN / Hamachi / ZeroTier** trên cả 2 máy → join cùng network ảo.
- Trên máy Server: **ipconfig** → tìm interface VPN → ghi nhớ IP dạng **26.x.x.x** (Radmin) / **25.x.x.x** (Hamachi).
- Client sửa **Session.cs** → trở về IP đó → rebuild.

A.5.3. Mở Firewall (Windows)

- Search **"Windows Defender Firewall with Advanced Security"** → **Inbound Rules** → **New Rule...**
- Chọn **Port** → thêm: **9000, 9001, 9002, 9003, 9011, 9102, 9103** → **Allow the connection**.

- (Cách nhanh test tạm: tắt firewall, nhớ bật lại sau).

B. Khởi động hệ thống

B.1. F5 lần đầu — 3 console mở đúng

Nhấn **F5**. Visual Studio sẽ build (nếu cần) rồi mở **3 cửa sổ Console**:

Console 1 — **AuthServer :9001**

```
[DB] DatabaseManager initialized
```

```
AUTH SERVER listening on :9001
```

Console 2 — **CanvasServer :9002**

```
[DB] DatabaseManager initialized
```

```
[DB] PersistenceQueue, AutoSaveService, and room state ready
```

```
CANVAS SERVER listening on :9002
```

Console 3 — **LoadBalancer :9000**

```
LOAD BALANCER listening on :9000
```

```
Auth pool: 1 backend(s)
```

```
Canvas pool: 1 backend(s)
```

```
AUTH • 127.0.0.1:9001
```

```
CANVAS • 127.0.0.1:9002
```

```
AUTH • 127.0.0.1:9011
```

← sẽ DOWN vì chưa start

```
CANVAS • 127.0.0.1:9003
```

← sẽ DOWN vì chưa start

Sau **~15 giây** (3 strike × ~5s mỗi probe), LB sẽ log:

```
[HEALTH] Auth 127.0.0.1:9011 probe 1/3 failed (...)
```

```
[HEALTH] Auth 127.0.0.1:9011 probe 2/3 failed (...)
```

```
[HEALTH] Auth 127.0.0.1:9011 -> DOWN (3 fails: ...)
```

💡 **Đây là demo Health Check 3-strike** — chụp màn hình đoạn này (sẽ dùng ở C.11).

Mỗi **15 giây**, LB in **[POOL]** snapshot trạng thái pool live:

```
[POOL]
Auth    127.0.0.1:9001 [UP]    conns=0/100 rooms=0 fails=0
Auth    127.0.0.1:9011 [DOWN]  conns=0/100 rooms=0 fails=3
Canvas  127.0.0.1:9002 [UP]    conns=0/100 rooms=0 fails=0
Canvas  127.0.0.1:9003 [DOWN]  conns=0/100 rooms=0 fails=3
```

B.2. Chạy 2 instance phụ (AuthServer #2 + CanvasServer #2) qua **.exe** + CLI args

Port đọc từ **CLI args đầu tiên** (**args[0]**).

AuthServer instance #2 (port 9011)

1. Mở **File Explorer** → **CanvasApp.AuthServer\bin\Debug**.
2. **Shift + chuột phải** vùng trống → **Open PowerShell window here**.
3. Gõ: **.\CanvasApp.AuthServer.exe 9011**

CanvasServer instance #2 (port 9003)

1. **CanvasApp.Server\bin\Debug** tương tự.
2. Gõ: **.\CanvasApp.Server.exe 9003** (peer port sẽ tự = 9103).

Sau **≤5s** (1 chu kỳ health check), LB log:

```
[HEALTH] Auth 127.0.0.1:9011 -> UP (recovered)
[HEALTH] Canvas 127.0.0.1:9003 -> UP (recovered)
```

→ **Demo Health Check recovery** — chụp màn hình (sẽ dùng ở C.11).

B.3. Script khởi động nhanh — **start-demo.ps1**

Tạo file **start-demo.ps1** ở root project:

```
$root = $PSScriptRoot
$auth = "$root\CanvasApp.AuthServer\bin\Debug\CanvasApp.AuthServer.exe"
$canv = "$root\CanvasApp.Server\bin\Debug\CanvasApp.Server.exe"
$lb = "$root\CanvasApp.LoadBalancer\bin\Debug\CanvasApp.LoadBalancer.exe"

Start-Process $auth -ArgumentList "9001"
Start-Process $auth -ArgumentList "9011"
Start-Sleep -Seconds 2
Start-Process $canv -ArgumentList "9002"
Start-Process $canv -ArgumentList "9003"
```

```
Start-Sleep -Seconds 2
Start-Process $1b

Write-Host "Pool started. Mở Client tay từ bin\Debug." -ForegroundColor Green
```

Chạy: **chuột phải file** → **Run with PowerShell** → toàn bộ pool 5 console bật lên trong ~5 giây.

C. Demo từng mục rubric (10đ)

#	Mục	Điểm	Section
1	App Logic + Socket Logic	5.0	C.1
2	I/O File + Network	0.5	C.2
3	Database	0.5	C.3
4	Thread / Đa luồng	0.5	C.4
5	Sign up / Sign in / OTP / Forgot Password	0.5	C.5
6	Multi Client	0.5	C.6
7	Multi Server	0.5	C.7
8	Cryptography	0.5	C.8
9	Demo LAN	0.5	C.9
10	Demo Internet (VPN)	0.5	C.10
11	Load Balancing	1.0	C.11
Tổng		10.0	

C.1. App Logic + Socket Logic (5.0đ)

Code evidence — 4 layer

Layer	File	Điểm nhấn
TCP Listener	CanvasApp.Server/Program.cs:88	<code>new TcpListener(IPAddress.Any, port)</code> + vòng <code>AcceptTcpClientAsync()</code>
Per-client handler	CanvasApp.Server/Program.cs:285-352	<code>HandleClient</code> — 1 Task/client, không block
Message dispatcher	CanvasApp.Server/Program.cs:354-728	<code>ProcessAsync</code> — switch case cho ROOM_, DRAW_, CHAT_, AUTH_, PING (12+ types)
Client socket	CanvasApp.Client/Network/CanvasClient.cs	<code>TcpClient</code> , <code>StreamReader/Writer</code> , line-delimited JSON, heartbeat 10s, reconnect

Layer	File	Điểm nhấn
		exponential backoff, generation counter
Domain logic	CanvasApp.Server/RoomManager.cs	Quản lý phòng, broadcast, undo stack, snapshot delta, atomic delete

Demo runtime — "1 vòng đời của 1 nét vẽ"

- Client #1:** login **demo1** → **Tạo phòng** → vẽ 1 nét bằng Pen.
- Client #2:** login **demo2** → Join cùng phòng → thấy nét vẽ realtime.
- Chỉ tay vào code** cho giảng viên thấy flow:
 - Client #1 [CanvasClient.SendDrawAsync] → JSON **DRAW_END** lên LB :9000
 - LB peek **MessageType** → forward về Canvas Server [Program.cs ProcessAsync case DRAW_END]
 - Server gọi [RoomManager.RecordDrawAction] gắn SeqNo + ActionId → enqueue vào [PersistenceQueue]
 - Server [BroadcastAsync] → tất cả client trong phòng (kèm **PEER_RELAY** sang server khác)
 - Client #2 [CanvasClient.ReceiveLoop] nhận → vẽ lên bitmap

Bằng chứng quan sát

- Console **CANVAS SERVER** log [ROOM] **DRAW_END** from demo1 seq=N
- Client #2 thấy nét vẽ trong < 100ms
- Wireshark filter **tcp contains "DRAW_"** → thấy chuỗi **DRAW_START** → **DRAW_MOVE** × N → **DRAW_END**

💡 **5đ này chiếm trọng số lớn nhất** — show kỹ flow + giải thích thiết kế single-join (**ROOM_RESOLVE** → direct TCP → một **ROOM_JOIN** duy nhất) thay vì double-join cũ.

C.2. I/O File + Network (0.5đ)

Code evidence

Tính năng	File
Đọc config JSON	LoadBalancer/Program.cs — <code>File.ReadAllText("appsettings.json")</code> + <code>JsonConvert.DeserializeObject</code>
Import ảnh nền	CanvasForm.cs — <code>OpenFileDialog</code> → <code>Image.FromFile()</code>
Export Canvas → PNG/JPEG	CanvasForm.cs <code>btnExport</code> — <code>SaveFileDialog</code> + <code>bitmap.Save(path, ImageFormat.Png/Jpeg)</code>
Image overlay sync (DRAW_IMAGE)	CanvasForm.Images.cs — <code>OpenFileDialog</code> → base64 → broadcast
Gửi file qua chat	CanvasForm.cs <code>SendChat</code> — <code>OpenFileDialog</code> (≤2MB) → Base64 → <code>CHAT_FILE</code>
TCP NetworkStream	CanvasClient.cs — <code>_client.GetStream()</code> → <code>StreamReader</code> (line-delimited JSON) + <code>StreamWriter</code>

Demo runtime

1. **Export ảnh:** Trong phòng → menu **Tệp** → **Xuất ảnh** → **.png** → mở file vừa lưu, đúng nội dung canvas.
2. **Import ảnh nền:** Menu **Tệp** → **Đặt ảnh nền** → chọn ảnh JPG → background canvas thay đổi.
3. **Image overlay:** Toolbar → nút Image → chọn ảnh → ảnh xuất hiện trên canvas, **kéo/scale** → Client #2 thấy ảnh di chuyển realtime (**DRAW_IMAGE_TRANSFORM**).
4. **Gửi file chat:** Ô chat → nút + → chọn PDF nhỏ → Client #2 thấy 📎 **filename.pdf (12 KB)** kèm link tải → click → tải về.
5. **(Tùy chọn) Wireshark:** Filter **tcp.port == 9000** → vẽ 1 nét → thấy gói TCP có payload JSON **{"type": "DRAW_END", ...}**. Chi tiết ở [phần D.1](#).

Bằng chứng quan sát

- File PNG/PDF mở được bằng Windows Photo / Adobe.
- Console server log **[CHAT_FILE] demo1 sent design.png (102400 bytes)**.

C.3. Database (0.5đ)

Code evidence

Layer	File
Connection factory	DatabaseManager.cs — OpenConnection() . Lưu ý: schema bootstrap nằm ở Database/schema.sql , không phải trong code.
DAOs (7 file)	DataAccess/ — UserDao, RoomDAO, RoomMemberDAO, DrawActionDAO, CanvasSnapshotDAO, ChatMessageDAO, plus AuthServer's OtpStore
Write-behind queue	PersistenceQueue.cs — BlockingCollection<Action> + 1 background consumer
Schema	Database/schema.sql — 7 bảng + index + UNIQUE(email)
Migration	UserStore.MigrateSchema — idempotent ALTER (vd add UNIQUE email) khi AuthServer start

Demo runtime

1. Mở **phpMyAdmin** → DB **canvasapp** → bảng **users** → có record từ register trước (**password_hash** dạng **\$2b\$12\$...**).
2. **Login lại** Client #1 → refresh bảng **users** → cột **last_login_at** cập nhật timestamp mới.
3. **Vẽ vài nét** → đợi ≤60s → bảng **canvas_snapshots** có row mới (**version**, **snapshot_data** LONGTEXT GZip-base64, **byte_size**).
4. **Tắt cả server và Client, F5 lại** → join lại phòng cũ → canvas vẫn còn nguyên (**RoomManager.EnsureCanvasLoaded** đọc snapshot + delta từ DB).
5. Show bảng **email_otp_codes** → cột **code_hash** dạng **\$2b\$10\$...** → OTP **không** lưu plaintext.

Bằng chứng quan sát

- phpMyAdmin show đủ 7 bảng + record cụ thể.

- Restart pipeline vẫn giữ data (canvas, room, history, snapshot).

C.4. Thread / Đa luồng (0.5đ)

Code evidence — 4 pattern

Pattern	File	Điểm nhấn
async/await + Task per client	Server/Program.cs	<code>AcceptTcpClientAsync</code> , <code>HandleClient</code> mỗi connection = 1 task
Task.Run + BlockingCollection	PersistenceQueue.cs	Producer-consumer write-behind queue, batch insert
System.Timers.Timer định kỳ	AutoSaveService.cs (snapshot mỗi 60s); HealthChecker.cs (probe mỗi 5s); PeerManager.cs (PEER_PING 15s)	Multiple timers chạy độc lập
Thread-safe collections + lock	RoomManager.cs	<code>ConcurrentDictionary</code> , <code>lock(list)</code> cho atomic delete, <code>SemaphoreSlim</code> (<code>_sendLock</code>) cho per-client serialize, <code>Interlocked</code> cho SeqNo

Demo runtime

1. Mở **3 Client** cùng lúc → cả 3 cùng vẽ → không lag/crash → mỗi client = 1 task riêng.
2. Console Canvas Server in `[PERSISTENCE]` `batched N actions in X ms` → write-behind queue chạy thread riêng.
3. Console LB in `[POOL]` snapshot mỗi 15s và `[HEALTH]` `probe` mỗi 5s → timer chạy độc lập với accept loop.
4. Console Canvas Server in `[PEER]` `PING -> canvas-9003 ... PONG received` → peer heartbeat task chạy riêng.

Bảng chứng quan sát

- 3 client vẽ đồng thời mượt mà.
- Multiple timer log cùng xuất hiện trong console.
- Atomic counters cho SeqNo: vẽ liên tục, mỗi action có SeqNo duy nhất, tăng đơn điệu.

C.5. Sign up / Sign in / OTP / Forgot Password (0.5đ)

Code evidence

Tính năng	File
UI Register	RegisterForm.cs — gửi <code>AUTH_SEND_OTP</code> rồi <code>AUTH_REGISTER</code> với OTP
UI Login	LoginForm.cs

Tính năng	File
UI Forgot Password	ForgotPasswordForm.cs — gửi <code>AUTH_FORGOT_SEND_OTP</code> rồi <code>AUTH_RESET_PASSWORD</code>
UI OTP entry	OtpVerifyForm.cs — shared modal cho cả 2 flow
Validate input	UserService.cs — <code>username ≥ 3</code> , <code>password ≥ 6</code> , email regex <code>≤ 100</code> chars
BCrypt + DB	UserStore.cs <code>HashPassword(req.Password, 12); Verify()</code>
OTP service	OtpService.cs — generate 6-digit, BCrypt cost 10, TTL 5min, single-use, rate-limit (<code>≤ 1/min</code> , <code>≤ 5/hour</code> per email)
SMTP gửi email	SmtplibEmailSender.cs
Token issue/verify	UserStore.cs <code>IssueToken/VerifyToken</code> — HMAC-SHA256 signed, TTL 24h, <code>FixedTimeEquals</code>
Session client	Session.cs — singleton <code>CurrentUser</code> , <code>Token</code>

Demo runtime

- Register:** Client → Register form → nhập `demo3 / 123456 / demo3@gm.uit.edu.vn` → bấm "Gửi mã OTP".
- Check email (hoặc console AuthServer in OTP nếu dev mode) → nhập OTP → submit → tài khoản tạo.
- Mở phpMyAdmin → bảng `users` → row mới, `password_hash` dạng `$2b$12$...` (không plaintext).
- Login sai password:** nhập `demo3 / wrong` → báo `Sai tên đăng nhập hoặc mật khẩu` (response time tương đương login đúng — constant-time, xem C.8).
- Login đúng:** → vào lobby → `last_login_at` update.
- Forgot Password:**
 - Click "**Quên mật khẩu**" trên LoginForm → nhập email `demo3@gm.uit.edu.vn` → bấm "Gửi mã".
 - Nhận OTP → nhập + new password → submit → thành công.
 - Login lại với password mới → vào lobby OK.
- OTP rate-limit demo:**
 - Bấm "Gửi mã OTP" lần 1 → success.
 - Bấm lần 2 ngay → reject `Vui lòng đợi 1 phút trước khi yêu cầu mã mới`.
 - (Test): 5 lần khác nhau trong 1h → lần 6 reject `Đã yêu cầu quá nhiều mã trong 1 giờ`.
- Persistent state:** vẽ trong phòng → close Client → mở lại → login → join phòng cũ → canvas vẫn còn.

Bảng chứng quan sát

- DB `users.password_hash` dạng `$2b$12$XYZ...` (60 ký tự).
- DB `email_otp_codes.code_hash` dạng `$2b$10$....`
- Rate-limit reject thấy ngay trên UI.

C.6. Multi Client (0.5đ)

Code evidence

- [RoomManager.BroadcastAsync](#) — gửi message đến tất cả client trong phòng.

- `Server/Program.cs` — gọi `BroadcastAsync` sau mỗi `DRAW_*`, `CHAT_MESSAGE`, `CHAT_FILE`, `ROOM_UPDATE`.
- Per-client send lock: `BroadcastService.cs` — `SemaphoreSlim` chống interleaving bytes giữa các broadcast đồng thời.

Demo runtime — 3 client cùng phòng

1. Mở **3 instance** `CanvasApp.Client.exe` (chuột phải project → Debug → Start New Instance × 3).
2. Login 3 user khác nhau (`demo1`, `demo2`, `demo3`) → cùng join 1 phòng.
3. Client #1 vẽ → #2 và #3 thấy realtime.
4. Client #2 gửi chat → cả #1 và #3 thấy (kể cả sender — đã fix chat self-echo từ 2026-05-22).
5. Right-panel **Danh sách thành viên** hiện 3 user với avatar màu khác nhau.
6. **Client #1 thoát** → #2, #3 thấy member list giảm còn 2 (broadcast `ROOM_UPDATE`).

Bằng chứng quan sát

- 3 client cùng vẽ — mượt, không miss stroke.
- Chat hiển thị đồng bộ ở mọi client.
- Member list cập nhật realtime.

C.7. Multi Server (0.5đ)

Code evidence — 3 service độc lập + peer mesh

Server	File chính	Port mặc định
AuthServer	<code>AuthServer.cs</code> , <code>Program.cs</code>	9001
CanvasServer	<code>Server/Program.cs</code> , <code>CanvasServer.cs</code>	9002 (client), 9102 (peer)
LoadBalancer	<code>LoadBalancer.cs</code>	9000
Peer mesh	<code>PeerManager.cs</code> (outbound) + <code>PeerHandler.cs</code> (inbound)	port = client port + 100

LB tách 2 pool dựa trên peek JSON message đầu:

- `LoadBalancer.cs` — `_authPool` (AUTH_), `_canvasPool` (ROOM_, DRAW_, CHAT_)

Demo runtime — 5 server đồng thời

1. F5 → 3 console mở (Auth :9001, Canvas :9002, LB :9000).
2. Chạy thử công 2 instance phụ (xem B.2):

```
. \CanvasApp.AuthServer\bin\Debug\CanvasApp.AuthServer.exe 9011
. \CanvasApp.Server\bin\Debug\CanvasApp.Server.exe 9003
```

3. Console LB hiện `Auth pool: 2 backend(s)` + `Canvas pool: 2 backend(s)` → **tổng 5 server**.
4. PowerShell:

```
netstat -ano | Select-String "LISTENING" | Select-String
"9000|9001|9002|9003|9011|9102|9103"
```

→ thấy 7 dòng **LISTENING** (3 client port + 2 peer port + Auth + LB).

Bằng chứng quan sát

- Banner LB hiện đủ pool 2+2.
- **netstat** confirm OS-level 5+ processes listening.
- Peer mesh log: **[PEER] handshake with canvas-9003 OK** ở console CanvasServer.

C.8. Cryptography (0.5đ)

Code evidence

Cơ chế	File	Chú thích
BCrypt password (cost 12)	UserStore.cs Register/Login	Hash 1 chiều + salt tự sinh
BCrypt room password (cost 10)	RoomManager.cs CreateRoom/UpdatePassword	Phòng riêng tư
HMAC-SHA256 token	UserStore.cs IssueToken/VerifyToken	<code>base64(payload).base64(HMAC(payload, secret))</code> — chống forge
Constant-time MAC compare	UserStore.cs FixedTimeEquals	Verify MAC không leak timing
TTL enforcement 24h	UserStore.cs VerifyToken	Reject token > 24h
Constant-time login	UserStore.cs Login	Chạy BCrypt dummy khi username không tồn tại → chống user-enumeration via timing
OTP BCrypt (cost 10)	OtpService.cs	OTP 6 số hash trước khi lưu
OTP rate-limit	OtpService.CheckRateLimit	≤1/email/min, ≤5/email/hour
AES-256-CBC + HMAC-SHA256 (opt-in)	AesHelper.cs	Encrypt-then-MAC, IV CT MAC base64
Message envelope	MessageCrypto.cs	Wrap <code>Message.Data</code> thành <code>{_enc, _v}</code> , <code>type + token</code> plaintext cho LB peek

Demo runtime

1. **BCrypt hash trong DB:** phpMyAdmin → bảng `users.password_hash` = `$2b$12$XYZ...` (60 ký tự, có salt) — không thể đảo ngược.
2. **HMAC token tamper:** copy token từ Session.cs lúc runtime → chỉnh 1 byte trong base64 phần signature → restart Client với token chỉnh → server reject **Token không hợp lệ**.
3. **Constant-time login:** dùng Stopwatch test login với username không tồn tại vs sai password — cả 2 đều ~100ms (BCrypt). **Trước fix:** username không tồn tại < 1ms → leak (đã sửa 2026-05-22).
4. **OTP rate-limit demo:** spam button "Gửi OTP" 2 lần liên tiếp → lần 2 reject **Vui lòng đợi 1 phút....**
5. **AES module demo** (LINQPad / Console temp project): xem [phần D.2](#).

Bảng chứng quan sát

- DB BCrypt hash dạng `$2b$12$....`
- Token tamper → reject visible trên client UI.
- OTP rate-limit reject visible.
- AES tamper → **CryptographicException: HMAC verification failed**.

Trạng thái mã hóa wire protocol

- ☒ **BCrypt password + OTP storage** — enabled
- ☒ **HMAC token signature** — enabled
- ☒ **AES message encryption** — module built, **chưa wire vào wire protocol** (traffic JSON vẫn plaintext). Để bật: uncomment `MessageCrypto.EncryptInPlace` ở 5 file (AuthClient, LobbyClient, CanvasClient, AuthHandler, Server.Program). Cố ý tắt để demo để thấy JSON qua Wireshark.

C.9. Demo LAN (0.5đ)

Code evidence — listener bind `IPAddress.Any` (0.0.0.0)

- [CanvasApp.Server/Program.cs:88](#) — `new TcpListener(IPAddress.Any, port)`
- [CanvasApp.AuthServer/AuthServer.cs:18](#) — `IPAddress.Any`
- [CanvasApp.LoadBalancer/LoadBalancer.cs:62](#) — `IPAddress.Any`

→ **Listen trên tất cả interface**, không chỉ loopback.

Demo runtime

1. **Máy A (Server):** PowerShell → `ipconfig` → đọc **IPv4 Address** (vd `192.168.1.42`).
2. Tắt firewall tạm hoặc add Inbound Rule (xem [A.5.3](#)).
3. Máy A chạy server (F5).
4. **Máy B (Client) — cùng Wi-Fi:** sửa `Session.cs` → `LB_HOST = "192.168.1.42"` → rebuild → chạy Client.
5. Login từ máy B → join phòng do máy A tạo → vẽ → máy A thấy nét vẽ realtime.

Bảng chứng quan sát

- PowerShell máy A: `netstat -ano | findstr :9000` → thấy connection **ESTABLISHED** với IP máy B.
- Canvas đồng bộ giữa 2 máy.

C.10. Demo Internet (VPN) (0.5đ)

Code evidence

- Cùng như [C.9](#) — không cần đổi code.

Demo runtime — qua VPN ảo (Radmin / Hamachi)

- Cài **Radmin VPN** trên cả máy A và máy B → tạo network mới → cả 2 join.
- Máy A: `ipconfig` → tìm interface **Radmin VPN** → IP dạng `26.x.x.x`.
- Máy A chạy server (F5).
- Máy B: sửa `Session.cs` → `LB_HOST = "26.x.x.x"` → rebuild → chạy Client.
- Máy B ở **mạng Internet khác hoàn toàn** (4G / Wi-Fi nhà khác) → vẫn login + vẽ được.

Bảng chứng quan sát

- Máy B ping `26.x.x.x` thành công.
- Canvas đồng bộ realtime qua Internet.

💡 Nếu giảng viên có 2 máy ở 2 mạng khác nhau → demo này gây ấn tượng nhất. Quay video sẵn dự phòng.

C.11. Load Balancing (1.0đ)

Code evidence

Tính năng	File	Điểm nhấn
Peek JSON đầu	<code>LoadBalancer.cs</code> <code>ReadOneLineAsync</code>	Đọc byte-by-byte có deadline, parse <code>Message.Type</code> , strip UTF-8 BOM
Routing 2 pool	<code>LoadBalancer.cs</code>	<code>_authPool</code> round-robin; <code>_canvasPool</code> room-affinity + least-loaded; <code>RouteForRoom</code> (sticky bind) vs <code>RouteForRoomReadOnly</code> (no bind on miss)
Sniff ROOM_CREATE_RESULT	<code>LoadBalancer.cs</code> <code>SniffAndForwardAsync</code>	Extract <code>Room.Id</code> → <code>RegisterRoom</code> mapping
Sniff ROOM_DELETE_RESULT	<code>LoadBalancer.cs</code> <code>SniffAndForwardAsync</code>	Trên <code>Success=true</code> → <code>UnregisterRoom</code> (giảm <code>RoomCount</code>)
Health check 3-strike	<code>HealthChecker.cs</code>	Probe TCP-connect mỗi 5s, fail 3 lần liên tiếp mới mark DOWN
Atomic counters	<code>ServerInfo.cs</code>	<code>Interlocked</code> cho <code>ActiveConnections</code> , <code>RoomCount</code> , <code>FailCount</code>
Config động	<code>appsettings.json</code>	Sửa <code>MaxConnections</code> không cần rebuild

Demo runtime — 6 bằng chứng cần show

Mỗi cái 1 screenshot/video. Đủ 6 → tròn 1.0đ.

Bằng chứng #1 — Banner LB khởi động đúng pool 2+2

- Bước B.1 + B.2.
- Kỳ vọng:

```
Auth pool: 2 backend(s)
Canvas pool: 2 backend(s)
```

Bằng chứng #2 — Health Check 3-strike DOWN

- Kill 1 backend (Ctrl+C console của nó), đợi ~15s.
- Console LB:

```
[HEALTH] Canvas 127.0.0.1:9002 probe 1/3 failed (...)
[HEALTH] Canvas 127.0.0.1:9002 probe 2/3 failed (...)
[HEALTH] Canvas 127.0.0.1:9002 -> DOWN (3 fails: ...)
```

- → **3-strike chống flap** khi mạng tạm chập 1 lần.

Bằng chứng #3 — Health Check UP (recovered)

- Restart backend đã chết:

```
.\CanvasApp.Server\bin\Debug\CanvasApp.Server.exe 9002
```

- Sau ≤5s, LB:

```
[HEALTH] Canvas 127.0.0.1:9002 -> UP (recovered)
```

Bằng chứng #4 — Auth round-robin

- Mở Client #1 → login user A → console LB:

```
[LB] 127.0.0.1:5xxxx -> Auth 127.0.0.1:9001 [UP] (first=AUTH_LOGIN)
```

- Mở Client #2 → login user B:

```
[LB] 127.0.0.1:5yyyy -> Auth 127.0.0.1:9011 [UP] (first=AUTH_LOGIN)
```

- 2 login đi 2 AuthServer khác.

Bằng chứng #5 — Canvas room-affinity stickiness

⚠ Cần dùng **Join by Invite Code** để LB peek được **ROOM_JOIN_BY_CODE** từ message đầu. Lobby gửi **ROOM_LIST** trước → LB không biết RoomId nếu join từ lobby card thông thường, sẽ dùng least-loaded.

- Client #1: login → **Tạo bảng trắng mới** → tạo room **ROOM-DEMO**.

```
[LB] ... -> Canvas 127.0.0.1:9002 (first=ROOM_LIST)
[ROUTE] room <RoomId> claimed by 127.0.0.1:9002 (sniff ROOM_CREATE_RESULT)
```

- Trong room → bấm **Sao chép link mời** → copy invite code.
- Client #2 (instance hoàn toàn mới): login → ngay khi vào lobby, paste invite code vào ô **Tham gia bằng mã** (KHÔNG click card).
- Console LB:

```
[ROUTE] room <RoomId> -> 127.0.0.1:9002 (rooms=1, sticky)
[LB] ... -> Canvas 127.0.0.1:9002 ... (first=ROOM_JOIN_BY_CODE)
```

→ Client #2 route về **cùng** Canvas Server **9002** với Client #1.

- Trong room: vẽ ở Client #1 → Client #2 thấy ngay → **canvas đồng bộ** → chứng minh affinity.

Bằng chứng #6 — Failover khi 1 Canvas Server chết

- Đảm bảo cả 2 CanvasServer đang chạy, có client đang vẽ.
- Tắt CanvasServer :9002 → LB log 3-strike DOWN (như #2) → drop mapping cho rooms host bởi 9002.
- Client trên 9002 disconnect → reconnect logic kích → tạo room mới → LB route về **9003**:

```
[LB] ... -> Canvas 127.0.0.1:9003 ...
```

- Bật lại **9002**: sau ≤5s, LB log **UP (recovered)** → pool đầy đủ trở lại.

Bonus demos (nếu còn thời gian)

Cross-server sync (đã có trong khi demo #5):

- 2 client → cùng phòng → khác Canvas Server (qua round-robin trước room-affinity) → Wireshark filter **tcp.port == 9102 or tcp.port == 9103** → thấy **PEER_RELAY** chạy giữa 2 canvas khi vẽ/chat.

Owner xóa room → **sync sang server khác**:

- Client #1 (owner) xóa room → console mọi peer log `[PEER_ROOM_DELETE] room X removed from local state`.
- Client #2 (đang ở lobby trên Canvas khác) → lobby refresh polling 3s → room biến mất.

Room cleanup ref-counting:

- Client cuối cùng leave/close → LB log:

```
[LB] ... closed (backend 127.0.0.1:9002 conns=0)
[ROUTE] room <RoomId> freed from 127.0.0.1:9002 (rooms=0)
```

MaxConnections cap:

- Sửa `appsettings.json` → `MaxConnections: 1` → restart LB.
- 3 Client cùng lúc → Client thứ 3 bị reject:

```
[LB] ... rejected - 127.0.0.1:9003 at MaxConnections
```

D. Phụ lục

D.1. Wireshark — show traffic thật

Mục tiêu: chứng minh client/server trao đổi JSON thật qua TCP, và 1 message đi qua LB → Auth/Canvas → response như thế nào.

Setup

- Cài **Wireshark + Npcap** (tick "**Install Npcap in WinPcap API-compatible Mode**" khi cài) — bắt buộc cho loopback 127.0.0.1.
- Mở Wireshark → chọn interface "**Adapter for loopback traffic capture**".
- Display Filter:

```
tcp.port in {9000 9001 9002 9003 9011 9102 9103}
```

Bảng filter chi tiết

Mục đích	Filter
Tất cả traffic CanvasApp	<code>tcp.port in {9000 9001 9002 9003 9011 9102 9103}</code>
Chỉ LB ↔ Client	<code>tcp.port == 9000</code>
Chỉ Auth pool	<code>tcp.port == 9001 or tcp.port == 9011</code>

Mục đích	Filter
Chỉ Canvas pool	<code>tcp.port == 9002 or tcp.port == 9003</code>
Peer mesh server-to-server	<code>tcp.port == 9102 or tcp.port == 9103</code>
Search 1 loại message	<code>tcp contains "ROOM_JOIN", tcp contains "AUTH_LOGIN", ...</code>
Chỉ DRAW events	<code>tcp contains "DRAW_"</code>
Chỉ chat	<code>tcp contains "CHAT_MESSAGE"</code>

Follow TCP Stream

1. Click bất kỳ packet → **Right-click** → **Follow** → **TCP Stream**.
2. Popup hiện toàn bộ JSON request/response như đoạn chat:
 - **Đỏ** = client gửi
 - **Xanh dương** = server trả lời
3. Export: **File** → **Export Specified Packets** → `.pcapng` để nộp kèm báo cáo.

4 scenario nên show

#	Demo	Filter / thao tác	Bằng chứng
1	Login qua LB → Auth	<code>tcp.port == 9000 or tcp.port == 9001</code> → Follow TCP Stream	Cùng <code>AUTH_LOGIN</code> xuất hiện 2 lần (LB là proxy plaintext)
2	Room affinity stickiness	<code>tcp.port == 9002 or tcp.port == 9003</code> — 2 client cùng vào 1 room	Cả 2 connection persistent đều đến cùng port
3	Peer mesh <code>PEER_RELAY</code>	<code>tcp.port == 9102 or tcp.port == 9103</code> — chat từ client A trên server khác client B	Có packet <code>{"type":"PEER_RELAY","data":{"...,"Inner":{"type":"CHAT_MESSAGE",...}}}</code>
4	Draw realtime	<code>tcp contains "DRAW_"</code> — vẽ 1 nét	Thấy chuỗi <code>DRAW_START</code> → <code>DRAW_MOVE × N</code> → <code>DRAW_END</code> + server-assigned <code>actionId</code>

Ví dụ thực tế (Auth flow):

```

{"type":"AUTH_LOGIN","data":{"Username":"ndln","Password":"123456"},"token":null}
client
{"type":"AUTH_LOGIN_RESULT","data":{"Success":true,"Token":"eyJ...", "User":
{"Id":1,...}}}
server
```

Lưu ý

- Loopback đôi khi miss packet đầu → stop & start capture lại.
- `tcp.port` hoạt động ở mức packet, không hỗ trợ regex nội dung → dùng `tcp contains "..."` để search text.
- Nếu thầy hỏi "sao đọc được rõ JSON vậy?" → trả lời: **plaintext JSON over TCP** (chưa bật AES — xem D.2).

D.2. AES module — demo encrypt + tamper

Module mã hóa AES-256-CBC + HMAC-SHA256 (Encrypt-then-MAC) sẵn sàng wire vào wire protocol.
Chưa enable trên live traffic — feature phòng khi cần.

Vị trí code

File	Vai trò
AesHelper.cs	AES-256-CBC + HMAC-SHA256 implementation
MessageCrypto.cs	Wrap/unwrap <code>Message.Data</code> thành <code>{ _enc: "...", _v: 1 }</code>
CryptoConfig.cs	Process-wide key + toggle (ENV <code>CANVASAPP_AES_KEY</code>)

Format ciphertext

Output `EncryptString` = Base64 của:

```
[ IV (16 bytes) ][ Ciphertext (PKCS7 padded) ][ HMAC-SHA256 (32 bytes) ]
```

- **IV**: random mỗi lần encrypt (CBC mode → semantic security)
- **HMAC** = SHA256(IV || Ciphertext) — verify TRƯỚC decrypt (Encrypt-then-MAC)
- **FixedTimeEquals**: compare MAC không leak timing

Demo 1 — Encrypt + Decrypt + Tamper (LINQPad/Console)

```
using CanvasApp.Common.Utills;
using System;

var key = AesHelper.KeyFromBase64("z9ZvBnQfX2YlS3o4nUvE3pK9XHN0V3FwS+rJqcXyB3o=");

// Encrypt
var plaintext = "{\"type\":\"AUTH_LOGIN\",\"data\":{\"Username\":\"ndln\",\"Password\":\"secret\"}}";
var ciphertext = AesHelper.EncryptString(plaintext, key);
Console.WriteLine($"Encrypted ({ciphertext.Length} chars base64):\n{ciphertext}");

// Decrypt
var decrypted = AesHelper.DecryptString(ciphertext, key);
Console.WriteLine($"Decrypted:\n{decrypted}");

// Tamper test
```

```
var tampered = ciphertext.Substring(0, ciphertext.Length - 4) + "AAAA";
try { AesHelper.DecryptString(tampered, key); }
catch (System.Security.Cryptography.CryptographicException ex)
{
    Console.WriteLine($"\\n✅ Tampering caught: {ex.Message}");
}
```

Output:

```
Encrypted (124 chars base64):
yIAjVxqxJK6Xq7M9...+8h2QYpZCk3W0eaPfg==

Decrypted:
{"type":"AUTH_LOGIN","data":{"Username":"ndln","Password":"secret"}}

✅ Tampering caught: HMAC verification failed (tampered or wrong key)
```

Đoạn này show cho thầy/cô:

- Mỗi lần encrypt cùng plaintext ra base64 khác → IV random (semantic security).
- Sửa 1 ký tự ciphertext → HMAC fail → message integrity được bảo vệ.

Demo 2 — Envelope trên Message

```
var msg = new Message("CHAT_MESSAGE",
    new { text = "hello secret world", userId = 1 },
    token: "eyJ...");

Console.WriteLine("Plaintext:\\n" + msg.ToJson());
// {"type":"CHAT_MESSAGE","data":{"text":"hello secret
world","userId":1},"token":"..."}

MessageCrypto.EncryptInPlace(msg, CryptoConfig.Key);
Console.WriteLine("\\nEncrypted:\\n" + msg.ToJson());
// {"type":"CHAT_MESSAGE","data":{"_enc":"yIAjVx...", "_v":1},"token":"..."}

MessageCrypto.DecryptInPlace(msg, CryptoConfig.Key);
```

Điểm mấu chốt:

- **type** và **token** **không** mã hóa → LB vẫn peek route được + AuthServer vẫn verify token được.
- Chỉ **data** (chứa password, chat, draw actions) được mã hóa.
- Versioned (**_v: 1**) để backward compatible khi nâng cấp format.

Demo 3 — So sánh trên Wireshark

- Chưa bật AES:

```
{"type":"AUTH_LOGIN","data":{"Username":"ndln","Password":"secret123"}}
```

- Sau khi bật AES:

```
{"type":"AUTH_LOGIN","data":{"_enc":"yIAjVxqxJK6X...","_v":1}}
```

→ Username/Password **không đọc được trên dây** nữa. **type** còn plaintext để LB peek route.

Wire AES vào prod (nếu cần demo full encrypted)

Chỉ cần sửa **AuthClient.cs** + **CanvasClient.cs** + **LobbyClient.cs** + **AuthHandler.cs** + **Server/Program.cs** — thêm 2 dòng mỗi chỗ:

Outbound (gửi):

```
if (CryptoConfig.EncryptionEnabled)
    MessageCrypto.EncryptInPlace(msg, CryptoConfig.Key);
await writer.WriteLineAsync(msg.ToJson());
```

Inbound (nhận):

```
var msg = Message.FromJson(line);
if (CryptoConfig.EncryptionEnabled && msg.Data != null)
    MessageCrypto.DecryptInPlace(msg, CryptoConfig.Key);
```

D.3. netstat — show số connection trực quan

```
# Đếm connection client đang giữ với từng canvas server
netstat -ano | findstr ":9002 :9003" | findstr ESTABLISHED

# Tổng connection theo port
netstat -ano | findstr ":900" | Measure-Object

# Xem các port đang LISTEN
netstat -ano | Select-String "LISTENING" | Select-String
"9000|9001|9002|9003|9011"

# Xem connection đến LB :9000
netstat -ano | Select-String ":9000"
```

Dùng để show: client #1 → :9002, client #2 → :9003 (round-robin) hoặc cả 2 → cùng :9002 (room-affinity sau khi join cùng phòng).

D.4. Troubleshooting

Triệu chứng	Nguyên nhân	Cách fix
LB banner báo Canvas pool: 0 rồi exit	Chưa start CanvasServer / port sai	Start CanvasServer trước, hoặc sửa appsettings.json của LB
Tất cả backend đều DOWN ngay khi LB lên	Firewall chặn loopback hoặc port đã bị process khác chiếm	netstat -ano findstr :9002 để xem ai giữ port. Kill process
Client login bị treo, LB log peek error: timeout	Client không gửi message JSON nào trong 5s	Check Session.cs đã đổi sang :9000 chưa; check Client connect đúng host
dropped – no first message in 5000ms	Client mở socket mà không gửi gì	Kiểm tra phía Client
Room-affinity không kick in (Client #2 về server khác)	Client gửi ROOM_LIST trước thay vì ROOM_JOIN_BY_CODE	Dùng đúng Join-by-Code flow; hoặc accept giới hạn và demo least-loaded
CanvasApp.Common.dll bị lock khi build	Có instance Client/Server còn chạy	Get-Process CanvasApp.* Stop-Process -Force
bin\Debug\ không có .exe	Chưa build / build Release	Ctrl+Shift+B rebuild; check Configuration là Debug
Login lỗi Token không hợp lệ hoặc đã hết hạn	Pre-2026-05-22 token không có signature	Re-login (HMAC migration)
OTP không gửi đến email	SMTP config sai trong AuthServer App.config	Check log AuthServer; xem OTP trong console (dev mode in plaintext)

D.5. Self-grading checklist

#	Mục	Điểm	Đã demo?	Bằng chứng chính
1	App Logic + Socket Logic	5.0	☐	3 client cùng phòng vẽ realtime + Wireshark DRAW_* flow
2	I/O File + Network	0.5	☐	Export PNG + gửi file chat + image overlay + (Wireshark TCP)
3	Database	0.5	☐	phpMyAdmin 7 bảng + restart server vẫn còn data
4	Thread / Đa luồng	0.5	☐	3 client vẽ + console batched persistence + timer logs
5	Sign up / Sign in / OTP / Forgot Password	0.5	☐	Register có OTP, DB hash BCrypt, OTP rate-limit reject

#	Mục	Điểm	Đã demo?	Bằng chứng chính
6	Multi Client	0.5	☐	3 client cùng phòng, vẽ + chat + member list sync
7	Multi Server	0.5	☐	5 server LISTENING (netstat) + peer mesh log
8	Cryptography	0.5	☐	DB <code>password_hash \$2b\$12\$...</code> + token tamper reject + AES demo
9	Demo LAN	0.5	☐	2 máy cùng Wi-Fi vẽ realtime
10	Demo Internet	0.5	☐	2 máy qua Radmin VPN từ 2 mạng khác
11	Load Balancing	1.0	☐	6 screenshot ở C.11
Tổng		10.0		

D.6. PowerShell quick commands

```
# Xem IP máy hiện tại (LAN/VPN demo)
ipconfig | Select-String "IPv4"

# Xem các port đang LISTEN (chứng minh 5 server đang chạy)
netstat -ano | Select-String "LISTENING" | Select-String "9000|9001|9002|9003|9011"

# Đếm số connection đến LB :9000
netstat -ano | Select-String ":9000" | Measure-Object

# Kill mọi instance CanvasApp (giải lock DLL trước khi rebuild)
Get-Process CanvasApp.* -ErrorAction SilentlyContinue | Stop-Process -Force

# Tail log file (nếu redirect console ra file)
Get-Content lb.log -Wait | Select-String "ROUTE|HEALTH|POOL"
```

D.7. Suggested presentation flow (~25 phút)

Bước	Show gì	Tool	Mục rubric
1	Mở đầu — kiến trúc 5 component	Slides + Docs/README.md	—
2	Login flow real-time	Wireshark Follow TCP Stream (port 9000 + 9001)	C.1, C.5, C.8
3	Register OTP + Forgot Password	UI + DB email_otp_codes + rate-limit reject	C.5
4	Tạo room + LB sniff	LB console <code>[ROUTE] claim on create</code>	C.7, C.11

Bước	Show gì	Tool	Mục rubric
5	Multi-client join cùng room	netstat (cả 2 connect cùng port 9002) + Wireshark	C.6, C.11
6	Vẽ broadcast realtime	Wireshark filter <code>tcp contains "DRAW_"</code> + UI 3 client	C.1, C.6
7	Image overlay drag/scale sync	UI 2 client (DRAW_IMAGE + DRAW_IMAGE_TRANSFORM)	C.2
8	Chat duplicate fix (history vs live)	Code: ChatHistory embedded in ROOM_JOIN_RESULT	C.1
9	Owner xóa room → sync sang server khác	Wireshark <code>tcp contains "PEER_ROOM_DELETE"</code>	C.7, C.11
10	Failover khi kill Canvas Server	LB console 3-strike + reconnect	C.11
11	DB persistence: restart server vẫn còn canvas	phpMyAdmin + UI	C.3
12	AES module demo	LINQPad: encrypt/decrypt/tamper	C.8
13	LAN/VPN demo (nếu có 2 máy)	2 laptop + Radmin VPN	C.9, C.10
14	Q&A — show source code	VS Code	—